CV

Miloš Vasić

AI Engineer — LLM infrastructure, autonomous agents, and the governance that makes them trustworthy.

- Email: milos85vasic@gmail.com
- Web: https://milosvasic.ru · https://vasic.digital
- GitHub: vasic-digital · HelixDevelopment · Server-Factory

Summary

AI/software engineer who builds AI-development systems end to end — from multi-provider LLM infrastructure and autonomous agents to the QA and governance layers that keep them honest. I don’t ship demos; I ship platforms. Over 15 years of professional engineering (since 2009) across mobile SDKs, real-time hardware integration, and distributed backends now converge on one focus: making autonomous AI development trustworthy at scale.

I architect fleets, not monoliths — large product applications standing on dozens of small, decoupled, independently-tested modules, every one of them inheriting a shared engineering Constitution and verified by an anti-bluff, evidence-gated QA discipline. Primary language Go, with Kotlin/KMP, TypeScript/React, Python, Swift, and Shell. Guiding principle, enforced mechanically rather than merely stated: a feature is done only when a real user can use it and there is captured evidence to prove it.

What I bring to the table: the ability to take an AI capability from a research idea to a governed, self-verifying, production-shaped system — LLM routing that proves each model actually works before trusting it, agents that debate and reach consensus instead of guessing, memory and RAG layers that don’t lose context, and a whole ecosystem wired so that “the tests are green” can never quietly mean “the feature is broken.”

Core competencies

- **AI / LLM systems:** multi-provider LLM abstraction (40+ providers), MCP tool integration, RAG, vector databases & embeddings, agent orchestration (headless CLI agents, graph workflows, multi-round debate/consensus), planning (HiPlan/MCTS/Tree-of-Thoughts), LLMOps, benchmarking (SWE-bench/HumanEval/MMLU), LLM verification, defensive-LLM guardrails, computer-vision + LLM-vision.
- **Backend engineering:** Go (Gin), gRPC + Protobuf, HTTP/3 (QUIC), WebSockets, distributed systems (incl. TLA+ formal specs), high-throughput REST services and concurrent workers.
- **Data:** PostgreSQL, SQLite, SQLCipher (encryption at rest), Redis, Neo4j, ClickHouse, object storage (MinIO/S3/GCS/Azure).
- **Frontend / cross-platform:** TypeScript/React (Tailwind, Redux Toolkit, i18next), Angular, Electron, React Native, Kotlin Multiplatform, Android/Android TV (Kotlin), iOS (Swift), Tauri/Rust.
- **Infra / DevOps:** Docker & Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/Parallels; CI/CD via GitHub Actions, Gradle, Make.
- **QA / quality engineering:** anti-bluff evidence-gated QA (HelixQA), Challenge harnesses with mutation gates, `go test -race`, visual-regression testing, ADB device testing, SonarQube, security scanning (semgrep/gosec/trivy/snyk/gitleaks/nancy).
- **Engineering governance:** Constitution-as-submodule; inheritance and propagation gates; documentation/coverage discipline across a 140+-repository fleet.

Selected projects

Governance & QA

- **HelixConstitution** — a universal engineering rulebook distributed as a Git submodule and inherited across 140+ repositories: engineering law shipped and version-pinned exactly like code. One submodule bump upgrades the rules for the entire fleet, and propagation gates literally grep every consuming repo for the required clause — each gate paired with a mutation meta-test that proves the gate itself isn't a sham. It turns "best practices people hope to follow" into inherited, auditable, mechanically-enforced anti-bluff law.
- **HelixQA** — anti-bluff QA orchestration (Go) built on one uncompromising rule: the bar is not "tests pass" but "users can use the feature." It runs written YAML test banks *and* fully-autonomous LLM-and-vision QA sessions that open the

real app, verify every documented feature, hunt for undocumented bugs across Android/Android TV/Web/Desktop, and refuse to score a PASS without captured runtime evidence — screenshots, logcat, video, stack traces — plus AI-fix-ready tickets.

AI development & LLM infrastructure

- **HelixAgent** — production-grade ensemble LLM service (Go/Gin) that refuses to trust a single model: it fans a prompt across many providers, runs structured multi-round debate (Proposal → Critique → Review → Synthesis), and routes by live verification scores with graceful fallback — all behind an OpenAI-compatible API with HA data layer, observability, and guardrails. *Go, Gin, PostgreSQL, Redis, Prometheus/Grafana/OpenTelemetry, MCP, Neo4j/ClickHouse/Kafka.*
- **HelixCode** — distributed AI development platform that divides work into intelligent, dependency-aware tasks across an SSH-managed worker fleet, then checkpoints and rolls back so nothing is ever lost when a task is interrupted; hardware-aware model selection and full plan/build/test/refactor lifecycle behind REST/CLI/TUI/MCP. *Go, Gin, PostgreSQL, Redis, SSH, MCP, llama.cpp/Ollama.*
- **HelixLLM** — one binary, six deployment modes: OpenAI- and Anthropic-compatible inference over HTTP/3 that scales from a laptop to a multi-host cluster, with local llama.cpp inference (CUDA/Metal/ROCm) and an auto-discovering, verification-scored cloud fallback chain that always degrades to a guaranteed local model. *Go, HTTP/3 QUIC, gRPC/SSE/Kafka, llama.cpp.*
- **LLMProvider / LLMOrchestrator / LLMsVerifier** — the LLM-infrastructure spine: one interface over 43 providers with circuit breakers, health monitoring, jittered-backoff retries and honest (no-hardcoded-fallback) model discovery; a thread-safe control plane that spawns and drives headless CLI agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) over a hybrid pipe+file protocol; and a verification source of truth whose mandatory “Do you see my code?” gate means only models proven to actually work are ever marked usable or exported.
- **HelixMemory / HelixSpecifier** — a unified cognitive-memory engine fusing four best-in-class backends (Mem0, Cognee, Letta, Graphiti) behind one interface with parallel search and cross-source re-ranking; and a spec-driven-development fusion engine that scales its own ceremony to the size of the work and backs specifications with multi-agent debate.
- **HelixTrack** — a free-world JIRA + Confluence alternative (flagship of the Helix-Track line): Go microservices with a unified action-routed API over HTTP/3, SQLCipher encryption at rest, and native Web/Desktop/Android/iOS clients.

Product-grade tools (vasic-digital utils)

- **Catalogizer** — multi-protocol, encrypted, self-hostable media collection management (Go/Gin + React), resilient to flaky network storage, built on 21 reusable submodules.
- **Courses-Creator** — markdown-to-video AI course pipeline with TTS and desktop/mobile/web players.
- **VisionEngine** — computer-vision + multi-provider LLM-vision UI perception with navigation graphs.
- **DocProcessor** · **Docs Chain** · **Herald** · **task_bridge** · **Vasic Digital Reusable Module Suite** — QA feature-mapping, content-hashed doc/DB sync, natural-language notifications, task/board sync, and the `digital.vasic.*` standard-library fleet.

Infrastructure automation (Server Factory)

- **Mail Server Factory** — declarative JSON → fully-provisioned, Dockerized mail servers across 12 connection types and 25 Linux distributions; reports 439 passing tests and a clean SonarQube gate.
- **Server Factory Core Framework**, **Qemu-Utils**, **Parallels-Utils** — the shared provisioning engine and VM-image tooling.

Languages & tools (quick list)

Go · Kotlin · Kotlin Multiplatform · TypeScript · JavaScript · Python · Swift · Java · Rust · Shell · PL/pgSQL · TLA+ · Gin · gRPC · HTTP/3 · React · Angular · Electron · React Native · PostgreSQL · SQLite · SQLCipher · Redis · Neo4j · ClickHouse · Docker · Kubernetes · Prometheus · Grafana · OpenTelemetry · QEMU · GitHub Actions · Gradle · Make

Experience

Software engineer since 2009, across the full development lifecycle — planning, development, team leading, and deployment. Full history below is sourced from the candidate's verified record (milosvasic.ru).

Full-time positions

- **SDK Developer — Harness** (harness.io), Belgrade, Serbia · 03/2020 – 12/2024. Lead developer on the family of SDKs for the company's Feature Flag division, focused on all major mobile platforms and beyond. Customers and partners included AWS, Google, and various banks. *Tech: Android, iOS,*

Flutter, React Native, TypeScript, JavaScript, Java, Kotlin, Swift, Go, Ruby.

- **Software Engineer — Leica Geosystems** (leica-geosystems.com), Heerbrugg, Switzerland · 02/2016 – 02/2020. Primarily iOS and Android engineering for Leica Geosystems' cutting-edge 3D scanners — real-time communication with the hardware, data processing, and synchronization. Partner: Autodesk. *Tech: Android, iOS, Java, Kotlin, Swift, C++.*
- **SDK Developer — Bosch** (bosch.rs), Belgrade, Serbia · 01/2010 – 01/2016. Lead SDK developer for the Connected Vehicles SDK project — real-time Bluetooth communication with the OBD2 bus, high-performance data processing and persistence. *Tech: Android, Java, Kotlin.*

Other engagements

- **TN-TECH** (tn-tech.co.rs), Novi Sad, Serbia · part-time, from 03/2017. Work for Globex Data (Canada and Switzerland) — Sekur (SekurMessenger), SekurMail, SekurSuite — and the BusRide platform. *Tech: Android, Java, Kotlin, C++, Qt.*
- **Increment Loop** (incrementloop.com), Belgrade, Serbia · part-time, from 09/2023. The Yuno application. *Tech: Android, Kotlin.*
- **Open-source / own organizations** — HelixTrack, Server Factory (Mail Server Factory, Parallels-Utils, Qemu-Utils), and Vasic Digital (Android-Toolkit, Network-Binder), detailed under Selected projects above.

Publications

- **Fundamental Kotlin** — self-published author; latest revised edition September 2022 (Fundamental Kotlin, 3rd Edition). Also authored for Packt Publishing (UK).

Education

- **M.Sc, Contemporary Information Technologies** — University Singidunum, Belgrade, Serbia · 2014.
- **B.Sc, Informatics and Computing** — University Singidunum, Belgrade, Serbia · 2008.